Programación Multiparadigma con Elixir
Modelos y Paradigmas de programación Universidad del Valle



I. INTRODUCCIÓN

Elixir es un lenguaje dinámico y funcional para la construcción de aplicaciones escalables y mantenibles. Elixir aprovecha la máquina virtual de Erlang, conocido por ejecutar sistemas de baja latencia, distribuidos y tolerantes a fallos, a la vez que se usa para el desarrollo web y el dominio de software integrado.
Este documento pretende hacer un análisis del lenguaje de programación Elixir enmarcado en los conceptos presentados en el curso Modelos y Paradigmas de Programación de la Universidad del Valle. Inicialmente se dará un vistazo general de las caracterısticas del lenguaje, posteriormente se discutirán los aspectos de Elixir que se asemejan a los modelos de programación vistos en el curso.

II. CARACTERÍSTICAS DEL LENGUAJE

En esta sección se presentan las caracterısticas generales del lenguaje: Tipos básicos, operadores básicos y reconocimiento de patrones.

II-A. Tipos básicos

- Entre los tipos básicos de Elixir encontramos:
- Enteros: 1, 0x1F
- Flotantes: 1.0
- Booleanos: true/false
- Átomos: :hola
- Strings: ”Hola mundo”
- Listas: [1 2 3]
- Tuplas: {1,2,3}

Generalmente existe la interrogante entre si usar lista o tuplas para resolver algún problema, para esto es necesario conocer las diferencias que existen entre estas estructuras de datos en Elixir. Por un lado, las listas en Elixir son almacenadas en memoria como listas enlazadas, lo que significa que cada elemento de la lista mantiene su valor y apunta al siguiente elemento hasta llegar al final. Esto significa que para acceder al tamaño de una lista necesitamos recorrerla completamente, es decir una operación lineal. Por otro lado, las tuplas son almacenadas contiguamente en la memoria. Esto significa que acceder al tamaño de la tupla ó

obtener un elemento por ındice es rápido. Sin embargo, actualizar o añadir elementos es costoso ya que se requiere crear una nueva tupla en memoria.

II-B.Operadores Básicos

Sobre enteros y flotantes:

- 1+2 Suma-Resta
- 2*3 Producto
- 10 / 2 (División Flotante)
- div(10, 2) (División Entera)

Sobre Strings:

- “a” <> “b” Concatenación

Sobre Booleanos:

- or o || and o &&

Sobre Listas:

- [1,2,3]++[4,5] = [1,2,3,5]
- [1,2,3]- -[2] = [1,3]

Comparación:
Los operadores presentados aquı retornan un booleano como resultado: ==, ! =, <=, >=, <, >.

II-C. Coincidencia de Patrones

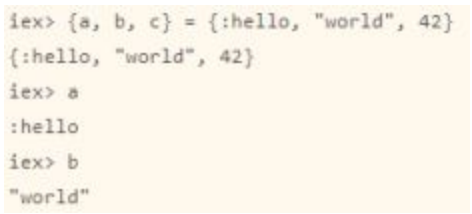
En Elixir se utiliza el operador = como operador de coincidencia (Match) y se usa ampliamente para emparejar patrones dentro de estructuras de datos. En la figura 1 se muestra un ejemplo básico del operador =.

Figura 1. Operador de Coincidencia

```
iex> 1 = x
1
iex> 2 = x
** (MatchError) no match of right hand side value: 1
```

El operador de coincidencia no sólo se usa para valores simples, también es útil para desestructurar tipos de datos más complejos, por ejemplo en la figura 2 se presenta el uso del operador = sobre tuplas.

Figura 2. Coincidencia de patrones sobre tuplas



III. ELIXIR VS PARADIGMAS DE PROGRAMACIÓN

En el Cuadro 1 se presenta un resumen de los paradigmas que son y no son compatibles con Elixir, en algunos casos el modelo es implementado a través de librerıas externas o extensiones del lenguaje. A continuación se muestra de forma detallada las razones y/o conceptos por los cuales un paradigma puede usarse o no en el lenguaje analizado.

Cuadro I PARADIGMAS IMPLEMENTADOS

Modelo	**¿Compatible con Elixir?**
Programación Declarativa	No
Programación Concurrente Declarativa	No
Programación con Estado	Si
Programación Concurrente por Paso de Mensajes	Si
Programación orientada a objetos	Si-Externo
Programación Relacional	No

III-A. Programación Declarativa

El concepto fundamental del paradigma declarativo es el de variable declarativa, una variable declarativa es un espacio de memoria donde se puede guardar un valor de cualquier tipo, que puede ser parcial, pero una vez está completo, no puedo ser cambiado, también son conocidas como variables de asignación única, es decir, pueden tomar cualquier valor durante una computación, sólo que una vez lo toma, ya no se puede cambiar.

En Elixir existe un concepto conocido como mutabilidad, Elixir no tiene variables mutables. La mutabilidad comúnmente se asocia al almacenamiento. En Elixir los valores almacenados no se pueden cambiar. Sin embargo, cuando se está escribiendo código en Elixir nos encontramos con la sorpresa de que al escribir lo siguiente no se produce un error.

```
Interactive Elixir (0.10.2)
iex(1)> animal="dog"
"dog"
iex(2)> animal="cat"
"cat"
iex(3)>
```



Lo primero que hay que entender es que el operador = no es un operador de asignación sino un operador de emparejamiento. Elixir puede hacer true(verdadero) el emparejamiento de animal = “dog” al unir la variable animal a “dog”. Lo que se esperaría de un lenguaje con variables inmutables es que al hacer animal = “cat” se produjera un error, sin embargo Elixir permite re enlazar (rebinding) las variables; ¿Entonces qué pasa con la premisa de que Elixir es un lenguaje inmutable? El re enlace no cambia el estado de un “objeto” en lo absoluto. El valor continúa en el mismo espacio de memoria, pero su etiqueta (nombre de variable) ahora apunta a otra posición de memoria, entonces la inmutabilidad es preservada.

En resumen, una variable en Elixir es un apuntador a un valor y el valor al que una variable puede apuntar es diferente, pero el valor nunca cambia. Entendiendo cómo funcionan las variables en Elixir, se puede observar que no se cumple explícitamente con el concepto de variable declarativa, el funcionamiento de estas variables puede asemejarse al concepto de celda en oz, utilizado en el modelo de programación con estado.

III-B. Programación Concurrente Declarativa

Elixir es considerado un lenguaje naturalmente concurrente, sin embargo el paradigma de programación concurrente declarativo requiere el concepto de variable declarativa y de hilos, como se mostró en la sección anterior no poseemos variables declarativas en Elixir, por lo tanto no hay programa- ción concurrente declarativa. En la sección de programación concurrente por paso de mensajes se mostrar a cómo funciona la concurrencia en el lenguaje analizado.

III-C. Programación Concurrente por Paso de Mensajes

Uno de los puntos fuertes de Elixir es su compatibilidad con la concurrencia. Gracias a la máquina virtual de Erlang la concurrencia se aplica fácilmente. El modelo de concurrencia se basa en Actores(procesos), un proceso que se comunica con otros procesos a través del paso de mensajes. Los procesos en Elixir no sólo son la base de la concurrencia, sino que también proporcionan los medios para crear programas distribuidos y tolerantes a fallos.

Caracterısticas de un proceso en Elixir:

- Corre sobre BEAM (Erlang Virtual Machine).
- Tiene una dirección (PID).

- Es un proceso completamente aislado.
- No comparte memoria.
- Se comunica con otros procesos por mensajes.
- Tiene un estado privado (puede ser modificado únicamente por funciones de cálculo privadas).
- Podrıa tener oyentes de mensajes privados (funciones de cálculo) para procesar los mensajes entrantes desde adentro y afuera.
- Tiene su propio colector de basura.

La forma m´as fácil de crear un proceso en Elixir es mediante la instrucción spawn que toma una función nombrada o anónima, cuando se crea un nuevo proceso se devuelve un identificador de proceso o PID, para identificarlo de manera única dentro de la aplicación.

Creemos un módulo y definamos una función:

```
defmodule Example do
  def add(a, b) do
    IO.puts(a + b)
  end
end
```

Para evaluar una función asíncronamente usamos spawn/3:
spawn (Example , : add , [2 , 3])

Para comunicarse los procesos dependen del envıo de mensajes. Existen dos componentes principales para esto: **send/2** y **receive**. La función send permite enviar mensajes dado un PID. Se usa receive para escuchar y hacer “match” con los mensajes, en caso de que no haya “match” la ejecución no se interrumpe.
La función **listen** es recursiva, ´esto permite que los procesos en Elixir manejen múltiples mensajes. Sin la recursión los procesos terminarıan después de recibir el primer mensaje.

```
defmodule Example do
  def listen do
    receive do
      {:ok, "hello"} -> IO.puts("World")
    end

    listen
  end
end
```

```
iex> pid = spawn(Example, :listen, [])
#PID<0.108.0>

iex> send pid, {:ok, "hello"}
World
{:ok, "hello"}

iex> send pid, :ok
:ok
```

III-D. Programación con Estado

La programación con estado en Elixir está´a íntimamente ligada con la programación por paso de mensajes, los procesos en Elixir pueden almacenar estados, dichos estados poseen las siguientes caracterısticas:

- Es un almacenamiento privado para el proceso.
- Únicamente puede ser modificado por el proceso
- Cuando el proceso muere, el estado vuelve a su valor inicial.

La manera más´as sencilla que provee Elixir para mantener estado son los Agentes, a continuación se ejemplificará el funcionamiento de los agentes en Elixir.

```
iex> {:ok, agent} = Agent.start_link fn -> [] end
{:ok, #PID<0.57.0>}
iex> Agent.update(agent, fn list -> ["eggs" | list] end)
:ok
iex> Agent.get(agent, fn list -> list end)
["eggs"]
iex> Agent.stop(agent)
:ok
```

El agente empieza con un estado inicial de una lista vacıa. Luego se actualiza el estado del agente añadiendo un nuevo elemento a la cabeza de la lista. El segundo argumento de Agent.update/3 es una función que toma el estado actual del agente como entrada y retorna el nuevo estado deseado. Finalmente, se obtiene la lista completa. El segundo argumento de Agent.get/3 es una función que toma el estado como entrada y retorna el valor que Agent.get/3 devolverá. Una vez se haya terminado con el agente, se puede invocar Agent.stop/3 para terminar el proceso. Mediante el uso de agentes elixir nos permite el estado explícito y el estado encapsulado dentro de una estructura. En la siguiente sección de programación orientada a objetos se mostrará una manera diferente de usar estado.

III-E. Programación orientada a objetos

El paradigma de programación orientada a objetos no está implementado directamente en Elixir, no obstante existen acercamientos al paradigma que se basan en el concepto de

módulos. Los módulos en Elixir son un mecanismo para agrupar funciones dentro de un espacio de nombres. En este documento no se hará hincapié en la descripción formal de los módulos, sin embargo se mostrará un ejemplo práctico de cómo estos se utilizan para modelar el paradigma orientado a objetos.

Construcción de una clase y un objeto defmodule Car do

```
defmodule Car do
  @default_state %{
    type: "Car",
    x: 0
  }

  def new(state \\ %{}) do
    spawn_link fn ->
      Map.merge(@default_state, state) |> run
    end
  end

  defp run(state) do
    receive do
      :drive ->
        new_x = state.x + 1
        new_state = Map.put(state, :x, new_x)

        run(new_state)
    end
  end
end
```

Para instanciar un objeto de la clase Car se hace lo siguiente:
car = Car . new(%{ color : "Red " })
send (car , : drive) #=> [Car] [Red] X . . 0 −> 1
send (car , : drive) #=> [Car] [Red] X . . 1 −> 2

La funcion new/1 es llamada así por convención. Esta función crea un nuevo proceso en Elixir usando spawn link/1, al generar un nuevo proceso se ha creado una área en la que se puede encapsular el estado.
Map.merge(@ defaultsatate ,state) |> *run*

Dentro del proceso recién creado, los atributos predeterminados se fusionan con los atributos pasados como argumentos; entonces la función recursiva run/1 es llamada.

La funcion run/1 es el nucleo del "objeto" y el estado se pasa una y otra vez de forma recursiva a run/1, encapsulando el estado como un argumento para la función. Cuando se requiere actualizar el estado se llama a run/1 con el nuevo estado.

Uno de los componentes claves para que esta implementación de objetos funcione es la función receive, cuando receive es llamada se bloquea el proceso actual y espera hasta que llegue un mensaje. Cuando se envía un mensaje al proceso este se desbloquea y ejecuta el código correspondiente al mensaje. En este caso se calcula un nuevo estado al incrementar x

en una nueva variable,actualizando x en el Map que representa el estado y luego se llama recursivamente a la función run/1.

Hasta el momento se han provisto algunos conceptos de POO tales como: “clases”, instanciación(objetos), estado encapsulado y comportamientos (métodos) que cambian el estado mediante paso de mensajes. Ahora se mostrará cómo se implementa el concepto de herencia en Elixir.

Hay una cuestión que resolver para implementar herencia, la herencia permite extender tipos (clases) con nuevo estado y comportamiento, pero ¿Cómo se pueden representar tipos y ´ subtipos sin clases?. Si se mira detenidamente la definición del modulo Car , uno de los valores por defecto fue un campo llamado type con un valor “Car”. En Elixir clases y tipos pueden ser representados con datos simples, como binarios (strings). Para modelar herencia de tipos se usarán datos para representar el tipo Car y el subtipo Truck.

```
defmodule Truck do
  def new(state \\ %{}) do
    spawn_link fn ->
      typed_state = Map.merge(%{type: "Truck"},
      state)
      parent = Car.new(typed_state)

      Map.merge(%{parent: parent}, typed_state)
      |> run
    end
  end

  def run(state) do
    receive do
      :offroad ->
        IO.puts "Going offroad."
        run(state)
      message ->
        send(state.parent, message)
        run(state)
    end
  end
end
```

En la función new/1, se está sobreescribiendo el valor de la propiedad type e instanciando un nuevo Car, que crea un nuevo proceso padre.
Map.merge(%{parent:parent} , typedstate) |> *run*
Es necesario guardar el proceso padre para poder delegar mensajes cuando el subtipo no pueda atenderlos directamente; entonces se llama a la función run/1
La función run/1 de Truck es similar a la de Car, solamente se agrega un nuevo mensaje :offroad al cual responder. Cuando Truck recibe un mensaje que no entiende, lo reenvía a su padre Car.

truck = Truck.new(%{ color : ” Blue ” })
send (truck , : drive) #=>[Truck] [Blue] X . . 0 −> 1
send (truck , : offroad) #=>Going offroad.

De esta manera el tipo Truck ha heredado todos los métodos del tipo Car. En los párrafos anteriores se expone una forma de implementar programación orientada a objetos con Elixir desde cero, pero en la investigación realizada se encontró una librería/módulo externo que permite POO de una forma más´ natural. Con el paquete encontrado la implementación de ´ clases y objetos se reduce así:

```
import OOP

class Person do
  var :name

  def say_hello_to(who) do
    what = "Hello #{who.name}"
    IO.puts("#{this.name}: #{what}")
  end
end

joe = Person.new(name: "Joe")
mike = Person.new(name: "Mike")
robert = Person.new(name: "Robert")

joe.say_hello_to(mike)     # Joe: Hello Mike
mike.say_hello_to(joe)     # Mike: Hello Joe
mike.say_hello_to(robert) # Mike: Hello Robert
robert.say_hello_to(mike) # Robert: Hello Mike

joe.set_name("Hipster Joe")
joe.name # => Hipster Joe
```

La herencia también está implementada con este paquete:

```
class Animal do
  var :name
end
class Dog < Animal do
  var :breed
end

snuffles=Dog.new(name:"Snuffles",breed:"Shih")
snuffles.name # => "Snuffles"
snuffles.breed # => "Shih"
```

O la herencia múltiple

```
class Human do
  var :name
end

class Horse do
  var :horseshoes_on?
end

class Centaur < [Human, Horse] do
end

john=Centaur.new(name:"John",horseshoes_on?:true)
john.name # => "John"
john.horseshoes_on? # => true
```

III-F. Programación Relacional

En cuanto a programación relacional, no se ha encontrado información que se asemeje al concepto de selección no determinística, lo más cercano con la programación relacional ´ obtenido es un módulo llamado ExKanren, que implementa programación relacional en Elixir basado en miniKanren. miniKanren es un lenguaje específico de dominio incorporado para la programación lógica. El núcleo de miniKanren es muy simple, solo con tres operadores lógicos y un operador de interfaz, se ha extendido proveyendo programación lógica con restricciones (implementada en ExKanren). Dada la poca documentación del módulo, sólo se presenta como un aparte informativo.

IV. CONCLUSIONES

En este an´alisis se ha logrado presentar a Elixir como un lenguaje multiparadigma, conociendo mediante ejemplos cómo encaja el lenguaje dentro de los modelos presentados en el curso de Modelos y Paradigmas de Programación. Se entiende entonces que Elixir es un lenguaje fuerte cuando de concurrencia se habla, el paradigma mejor soportado es el de concurrencia por paso de mensajes, ya que en Elixir todo es considerado un proceso. El paradigma de programación por paso de mensajes permite dar vida a la implementación de la programación orientada a objetos de una forma aparentemente sencilla y por ende da cabida a la programación con estado. Los modelos: declarativo, concurrente declarativo y relacional no se encuentran dentro de el lenguaje. Si se pudiese especificar qué variables pueden o no ser re enlazadas el modelo de programación declarativo sería soportado de forma natural y simple en Elixir.

Referencias? Siempre es importante mencionarlas....
Me gustó mucho el trabajo y la forma de analizar el lenguaje... Veo que el curso fue bien aismilado...